

DataCo Supply Chain

Data Warehouse — Technical Documentation

Built with PostgreSQL • Python • Power BI

Ismail — DEPI Portfolio Project — 2024

Dataset	DataCo Supply Chain (Kaggle)
Rows	180,518 order line items
Date Range	January 2015 — January 2018
Database	PostgreSQL 16
Schemas	staging · warehouse · marts
Dimensions	5 (customer, product, date, geography, shipping)
Fact Table	fact_orders — ~180,518 rows
Data Marts	3 materialized views
Output	Power BI dashboards

1. Project Overview

The DataCo Supply Chain Data Warehouse is a portfolio project demonstrating end-to-end data engineering on a real-world supply chain dataset. The project transforms a flat CSV file containing 180,518 order records into a purpose-built analytical platform using a three-layer architecture: staging, warehouse, and marts.

The warehouse is designed to answer business questions about delivery performance, product profitability, and monthly revenue trends — and connects directly to Power BI for interactive reporting.

1.1 Business Questions Answered

- Which markets and shipping modes have the highest late delivery rates?
- Which product categories and departments generate the most profit and best margin?
- What is the month-over-month revenue and profit trend per market?
- How does the shipping delay compare across shipping modes?
- Which customer segments drive the most revenue?

1.2 What Was Built

Layer	Schema	Objects	Technology
Ingestion	staging	stg_orders (180,518 rows)	Python · pandas · sqlalchemy
Warehouse	warehouse	5 dimensions + 1 fact table	PostgreSQL 16 · pgAdmin
Marts	marts	3 materialized views	PostgreSQL · SQL
Reporting	—	Delivery · Profitability · Revenue	Power BI

2. Dataset

2.1 Source

The DataCo Supply Chain dataset was sourced from Kaggle (saicharankomati/dataco-supply-chain-dataset). It represents a synthetic global supply chain company — DataCo Global — operating across five markets: LATAM, Europe, Pacific Asia, USCA, and Africa.

Property	Value
Source	Kaggle — DataCo Supply Chain Dataset
File size	~96 MB CSV
Original rows	180,519
After dedup	180,518 (1 duplicate removed)
Columns	53 (51 after dropping useless columns)
Date range	January 2015 — January 2018
Markets	LATAM · Europe · Pacific Asia · USCA · Africa
Order statuses	COMPLETE · PENDING · PROCESSING · SUSPECTED_FRAUD · CANCELED · ON_HOLD

2.2 Column Categories

Category	Columns	Destination
Payment & Delivery	Type, Days shipping real/scheduled, Delivery Status, Late risk, Shipping Mode, Order Status	fact_payment_delivery
Customer	Customer Id, Name, Segment, City, State, Country, Zipcode	dim_customer
Order Financials	Order Id, Item Id, Quantity, Price, Discount, Total, Sales, Profit, Profit Ratio (measures)	fact_order_financials
Product	Product Id, Name, Category, Department, Price, Status	dim_product
Geography	Market, Region, Country, City, State, Latitude, Longitude	dim_geography
Dates	Order date, Shipping date	dim_date
Dropped	Product Description (100% NULL), Product Image (URLs only)	Removed at ingestion

3. Architecture

The project follows a classic three-layer data warehouse architecture. Each layer has a single clearly defined responsibility and passes clean, validated data to the next layer.

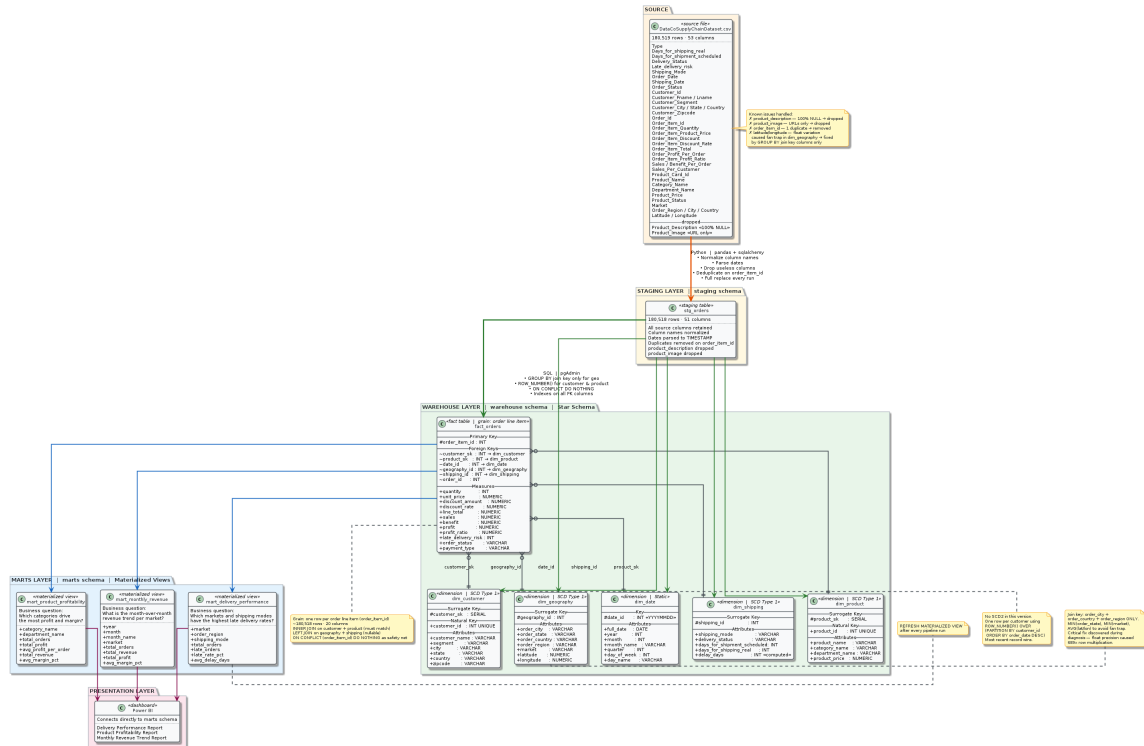


Figure 1 — Complete project architecture showing all layers, tables, and data flow

3.1 Layer 1 — Staging

The staging layer is a raw landing zone. The Python script reads the full CSV and loads it into PostgreSQL without any business logic transformations. Only cosmetic operations are performed: column name normalization, date parsing, dropping useless columns, and deduplication on `order_item_id`.

Key design choice: `if_exists='replace'` drops and reloads the entire staging table on every run. This makes the pipeline fully idempotent — safe to re-run at any time with the same result.

3.2 Layer 2 — Warehouse (Star Schema)

The warehouse layer implements a star schema — the industry standard for analytical data warehouses. One central fact table records every order event. Five dimension tables describe the context of each event: who, what, when, where, and how.

The fact table uses foreign keys to point to dimension tables rather than repeating descriptive text. This eliminates redundancy, enforces referential integrity, and makes analytical joins fast and consistent.

3.3 Layer 3 — Marts

The marts layer contains three materialized views that pre-aggregate the warehouse data to answer specific business questions. Materialized views store computed results on disk — analytical queries run in milliseconds instead of scanning 180,518 rows on every request.

Power BI connects directly to the marts schema using DirectQuery or Import mode.

4. Star Schema Design

4.1 Fact Table — fact_orders

Grain: One row per order line item (order_item_id). Every measurable attribute of each order event is stored here as a numeric measure. Descriptive context is accessed by joining to dimension tables.

Column	Type	Description
order_item_id	INT PK	Primary key — one row per order line item
order_id	INT	Parent order (one order can have many line items)
customer_sk	INT FK	Surrogate key → dim_customer
product_sk	INT FK	Surrogate key → dim_product
date_id	INT FK	YYYYMMDD integer → dim_date
geography_id	INT FK	→ dim_geography
shipping_id	INT FK	→ dim_shipping
quantity	INT	Units ordered in this line item
unit_price	NUMERIC	Product price before discount
discount_amount	NUMERIC	Discount applied in dollars
discount_rate	NUMERIC	Discount as a decimal ratio 0-1
line_total	NUMERIC	Total after discount
sales	NUMERIC	Actual revenue for this line item
benefit	NUMERIC	Gross earnings
profit	NUMERIC	Net profit
profit_ratio	NUMERIC	Profit divided by sales
late_delivery_risk	INT	1 = at risk of late delivery, 0 = on time
order_status	VARCHAR	COMPLETE, CANCELED, SUSPECTED_FRAUD etc.
payment_type	VARCHAR	DEBIT, TRANSFER, CASH, PAYMENT

4.2 Dimension Tables

Dimension	SCD Type	Grain	Key Attributes
dim_customer	SCD1	One row per customer	customer_id, name, segment, city, state, country
dim_product	SCD1	One row per product	product_id, name, category, department, price
dim_date	Static	One row per date	date_id (YYYYMMDD), year, month, quarter, day_name
dim_geography	SCD1	One row per city/country/region	market, region, city, lat, lon
dim_shipping	SCD1	One row per shipping combination	shipping_mode, delivery_status, delay_days

4.3 Dimension Details

dim_date — Static

Built by extracting every unique order date and pre-computing all analytical date parts. Date ID uses YYYYMMDD integer format (e.g. 20160315) — faster to join than date types and immediately human-readable.

dim_geography — SCD Type 1

Groups by join key columns only (order_city, order_country, order_region). Latitude and longitude use AVG() rather than being included in GROUP BY — float precision variations would otherwise create hundreds of duplicate geography rows for the same city, causing a fan trap that multiplies fact rows. This was discovered and fixed during the project.

dim_shipping — SCD Type 1

Includes a computed column `delay_days` = `days_for_shipping_real` minus `days_for_shipment_scheduled`. Positive values mean late delivery, zero means on time, negative means early arrival.

dim_customer — SCD Type 1

Uses `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC)` instead of `DISTINCT`. `DISTINCT` does not work correctly because `customer_zipcode` is 86% NULL — the same customer appears with and without a zipcode across different orders, creating multiple `DISTINCT` rows for the same person. `ROW_NUMBER` picks the most recent record only.

dim_product — SCD Type 1

Same `ROW_NUMBER()` pattern as `dim_customer`. One row per `product_id` using the most recent order record. No SCD Type 2 in this version — the most current product attributes are used for all historical and current analysis.

Note: SCD Type 2 (historical tracking) for dim_customer and dim_product is planned as a future enhancement. The current SCD1 implementation is intentional for simplicity and Power BI compatibility.

5. Data Quality Issues Discovered and Fixed

Three significant data quality issues were discovered during the project. Each was diagnosed systematically using SQL queries and fixed before the fact table was loaded.

5.1 Duplicate order_item_id in Source CSV

Property	Detail
Discovery	Python deduplication step: 180,519 rows reduced to 180,518
Root cause	One order_item_id appeared twice in the source CSV
Impact	Primary key violation on fact table insert
Fix	drop_duplicates(subset=['order_item_id'], keep='last') in Python
Layer fixed	Python ingestion — before staging load

5.2 Float Precision Fan Trap in dim_geography (Critical)

Property	Detail
Discovery	Diagnosis query: single order items matching up to 689 rows in the fact join
Root cause	latitude and longitude included in DISTINCT — float variations (40.7128 vs 40.7127) created hundreds of geographies
Impact	Each fact row was multiplied 689 times — primary key violation on insert
Fix	GROUP BY on join key columns only (order_city, order_country, order_region). Use AVG(latitude), AVG(longitude)
Layer fixed	dim_geography SQL script

```
-- Wrong: float columns in deduplication key cause fan trap
SELECT DISTINCT order_city, order_country, order_region, latitude, longitude ...

-- Correct: GROUP BY join key only, aggregate the floats
SELECT order_city, order_country, order_region, AVG(latitude), AVG(longitude)
FROM staging.stg_orders GROUP BY order_city, order_country, order_region
```

5.3 Residual Duplicates After First Geography Fix

Property	Detail
Discovery	Diagnosis query still returned 3 matches per order item after first fix
Root cause	order_state and market still included in GROUP BY — slight variations in these columns created multiple rows for each geography
Impact	Remaining duplicate fact rows — still 3x multiplication
Fix	Remove order_state and market from GROUP BY entirely. Use MIN(order_state) and MIN(market) as aggregate functions
Layer fixed	dim_geography SQL script — second revision

6. Project Structure

File / Folder	Purpose
python/load_staging.py	Python ingestion script — reads CSV, cleans, deduplicates, loads to staging
sql/00_drop_all.sql	Clean reset — drops all tables, views, and schemas
sql/02_dimensions/dim_date.sql	Static date dimension — YYYYMMDD primary key, pre-computed date parts
sql/02_dimensions/dim_geography.sql	Geography dimension — GROUP BY join key only, AVG coordinates
sql/02_dimensions/dim_shipping.sql	Shipping dimension — includes computed delay_days column
sql/02_dimensions/dim_customer.sql	Customer dimension — ROW_NUMBER deduplication by customer_id
sql/02_dimensions/dim_product.sql	Product dimension — ROW_NUMBER deduplication by product_id
sql/03_facts/00_diagnosis_check.sql	MANDATORY — run before fact load. Must return zero rows
sql/03_facts/fact_orders.sql	Central fact table — 180,518 rows, 20 columns, 5 FK indexes
sql/04_marts/marts.sql	Three materialized views for Power BI reporting
sql/sanity_check.sql	Full validation queries — run after complete pipeline
docs/DataCo_DWH_Architecture.png	PlantUML architecture diagram showing all layers
docs/DataCo_DWH_Architecture.puml	PlantUML source file — edit and regenerate diagram
docs/DataCo_DWH_Documentation.pdf	This documentation file
requirements.txt	Python dependencies: pandas, sqlalchemy, psycpg2-binary
README.md	Project overview, run instructions, design decisions
data/PUT_CSV_HERE.txt	Placeholder — place DataCoSupplyChainDataset.csv here

7. How to Run the Project

7.1 Prerequisites

Tool	Version	Purpose
PostgreSQL	14+	Database engine
Python	3.9+	Ingestion script
pgAdmin	Any	Running SQL scripts
pandas	2.0+	CSV reading and transformation
sqlalchemy	2.0+	Python-PostgreSQL bridge
psycopg2	2.9+	PostgreSQL driver for Python

7.2 Step-by-Step Run Order

Step 1 — Create the database

```
CREATE DATABASE dataco_dwh;
```

Step 2 — Set up Python environment

```
cd C:\Users\EGY10\dataco-dwh
```

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

Step 3 — Configure load_staging.py

Open python/load_staging.py and update these two lines:

```
DB_PASSWORD = "your_actual_password"
```

```
CSV_PATH = r"C:\path\to\DataCoSupplyChainDataset.csv"
```

Step 4 — Run the Python staging load

```
python python\load_staging.py
```

Expected: 180,518 rows loaded into staging.stg_orders. Script prints row count confirmation.

Step 5 — Run SQL scripts in pgAdmin (exact order)

Order	Script	What it does
1	sql/02_dimensions/dim_date.sql	Build static date dimension
2	sql/02_dimensions/dim_geography.sql	Build geography dimension — check returns zero rows
3	sql/02_dimensions/dim_shipping.sql	Build shipping dimension
4	sql/02_dimensions/dim_customer.sql	Build customer dimension
5	sql/02_dimensions/dim_product.sql	Build product dimension

6	sql/03_facts/00_diagnosis_check.sql	MANDATORY — must return zero rows before proceeding
7	sql/03_facts/fact_orders.sql	Build fact table with indexes
8	sql/04_marts/marts.sql	Create three materialized views
9	sql/sanity_check.sql	Final validation — confirm all counts and distributions

7.3 Expected Final Row Counts

Table	Expected Rows	Notes
staging.stg_orders	180,518	Full dataset after deduplication
warehouse.dim_customer	~12,000	One row per unique customer
warehouse.dim_product	~100	One row per unique product
warehouse.dim_date	~1,400	One row per unique order date
warehouse.dim_geography	~3,000	One row per city/country/region
warehouse.dim_shipping	~20	One row per shipping combination
warehouse.fact_orders	~180,518	Must match staging row count
Orphaned rows	0	MUST be zero — check with sanity_check.sql

8. Data Marts

Three materialized views serve as the analytical layer. Power BI connects to these views directly. Each mart pre-aggregates the warehouse data to answer a specific business question.

8.1 mart_delivery_performance

Business question: Which markets and shipping modes have the highest late delivery rates and longest delays?

Column	Description
market	Global market (LATAM, Europe, Pacific Asia, USCA, Africa)
order_region	Sub-region within the market
shipping_mode	Standard Class, First Class, Second Class, Same Day
delivery_status	Late delivery, On time, Advance shipping, Canceled
total_orders	Total number of orders in this group
late_orders	Count of orders with late_delivery_risk = 1
late_rate_pct	Percentage of late orders (0.0 to 100.0)
avg_delay_days	Average delay in days (positive = late, negative = early)

8.2 mart_product_profitability

Business question: Which product categories and departments generate the most profit and best margin?

Column	Description
department_name	Store department (Fan Shop, Apparel, Golf, etc.)
category_name	Product category (Fishing, Cleats, Cardio Equipment, etc.)
total_orders	Number of order line items in this category
total_units_sold	Sum of all quantities sold
total_revenue	Sum of all sales amounts
total_profit	Sum of all profits
avg_profit_per_order	Average profit per order line item
avg_margin_pct	Average profit ratio as percentage
avg_discount_pct	Average discount rate as percentage

8.3 mart_monthly_revenue

Business question: What is the month-over-month revenue and profit trend per market?

Column	Description
--------	-------------

year	Calendar year (2015, 2016, 2017, 2018)
month	Month number (1-12)
month_name	Month name (January, February, etc.)
quarter	Quarter number (1-4)
market	Global market
total_orders	Count of distinct parent orders
total_line_items	Count of order line items
total_revenue	Sum of sales
total_profit	Sum of profits
avg_margin_pct	Average profit margin percentage
avg_discount_pct	Average discount rate percentage
late_deliveries	Count of late delivery risk orders

Refresh all marts after every pipeline run:

```
REFRESH MATERIALIZED VIEW marts.mart_delivery_performance;
```

```
REFRESH MATERIALIZED VIEW marts.mart_product_profitability;
```

```
REFRESH MATERIALIZED VIEW marts.mart_monthly_revenue;
```

9. Design Decisions

9.1 Why GROUP BY join key only for dim_geography

The fact table joins to dim_geography on three columns: order_city, order_country, and order_region. Including any additional columns in the deduplication key — especially float columns like latitude and longitude — creates multiple geography rows for the same logical location. This causes a fan trap that multiplies fact rows by hundreds, resulting in a primary key violation on the fact load. Deduplicating on the join key alone guarantees exactly one geography row per fact row.

9.2 Why ROW_NUMBER instead of DISTINCT for customers and products

DISTINCT deduplicates on the full row combination of all columns selected. Since customer_zipcode is 86% NULL in this dataset, the same customer appears with a zipcode on some orders and NULL on others — producing two DISTINCT rows for the same customer. ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) guarantees exactly one row per customer regardless of attribute variation, always using the most recent record as the source of truth.

9.3 Why INNER JOIN for customer and product, LEFT JOIN for geography and shipping

Customer and product are business-critical dimensions — an order without a customer or product is meaningless and represents a data quality issue. INNER JOIN enforces this constraint and surfaces problems immediately. Geography and shipping are enrichment dimensions — an order still has meaningful revenue and profit even if the geography or shipping row is not found. LEFT JOIN preserves all orders and sets those FK columns to NULL rather than silently dropping rows.

9.4 Why ON CONFLICT DO NOTHING on the fact table

Even after the diagnosis check passes, ON CONFLICT DO NOTHING acts as a permanent safety net ensuring the fact load never fails on a duplicate key. It is a standard practice in production pipelines and adds no meaningful overhead to the insert operation.

9.5 Why no SCD Type 2 in this version

SCD Type 2 adds significant complexity: surrogate keys, effective dates, is_current flags, temporal joins in the fact load, and upsert stored procedures. This version deliberately uses SCD Type 1 (latest value wins) to keep the warehouse simple, fast to build, and directly compatible with Power BI without complex temporal join logic. SCD Type 2 will be added in a future iteration for dim_customer and dim_product.

9.6 Why materialized views for marts

Materialized views store pre-computed aggregation results on disk. Analytical queries against the marts run in milliseconds instead of re-scanning 180,518 fact rows on every request. The trade-off is that they must be refreshed after each pipeline run — a deliberate and acceptable choice for this batch-oriented

pipeline.

10. Key Findings

Analysis of the full 180,518 row dataset reveals several significant operational insights for the DataCo Global supply chain.

10.1 Delivery Performance

- 54.9% of all orders carry a late delivery risk — the single largest operational concern in the dataset
- Second Class shipping averages +2.07 days late despite being a paid upgrade over Standard Class
- Standard Class is the most reliable shipping mode, averaging nearly on-time delivery (-0.02 days)
- Same Day shipping performs consistently but represents only 5.7% of all orders

10.2 Product Profitability

- Fishing is the most profitable product category by total profit across the dataset
- Cardio Equipment leads in average profit per order despite not having the highest total revenue
- Cleats and Water Sports are consistently in the top five most profitable categories
- Some categories show negative average profit — indicating systematic over-discounting

10.3 Revenue Trends

- LATAM and Europe are the top two markets, virtually tied at approximately \$575K in total revenue
- Pacific Asia is third at approximately \$463K — significant gap from the top two markets
- Africa is the smallest market at approximately \$122K — potential growth opportunity
- Revenue shows consistent seasonal patterns with peaks in Q4 across all markets

10.4 Fraud and Risk

- 2.1% of orders (approximately 3,800) are flagged as SUSPECTED_FRAUD
- Suspected fraud orders show above-average profit per order — an unusual pattern worth investigating
- High-value orders are disproportionately represented in the fraud-flagged category

11. Future Enhancements

Enhancement	Description	Priority
SCD Type 2	Add effective_from, effective_to, is_current to dim_customer and dim_products to track historical changes	High
Incremental loading	Add watermark table to load only new records on each pipeline run rather than full replace	High
dbt	Replace raw SQL files with dbt models for version control, testing, and automation	Medium
Data quality tests	Add automated assertions after each pipeline step — null checks, range validation, referential integrity	Medium
Cloud deployment	Deploy PostgreSQL to AWS RDS and schedule pipeline runs with AWS Lambda on EventBridge	Medium
Airflow scheduling	Add Apache Airflow DAG to orchestrate and monitor pipeline runs automatically	Low
Additional marts	Add mart_customer_segments and mart_fraud_analysis for deeper business insights	Low
Power BI deployment	Publish Power BI reports to Power BI Service with scheduled refresh	Low

12. Tech Stack

Layer	Tool	Version	Purpose
Database	PostgreSQL	16	Primary data warehouse engine
Query Tool	pgAdmin	4	Running SQL dimension and fact scripts
Ingestion	Python	3.11	CSV ingestion and transformation script
Data library	pandas	2.0+	CSV reading, column cleaning, deduplication
DB bridge	SQLAlchemy	2.0+	Python-to-PostgreSQL connection
DB driver	psycopg2-binary	2.9+	PostgreSQL adapter for Python
Visualization	Power BI Desktop	Latest	Dashboard creation and business reporting
Diagram	PlantUML	1.2024	Architecture diagram generation
OS	Windows 11	—	Development environment

DataCo Supply Chain Data Warehouse — Technical Documentation

Ismail — DEPI Portfolio Project — Built with PostgreSQL and Python